

IFT2015-H26 - Quiz #3 - 5 février 2026

Nom:

Prénom:

Matricule:

Directives:

- Répondez directement sur cette feuille
- Aucune documentation ni appareil électronique permis.

(2 points) Question 1 : Parmi les affirmations suivantes, lesquelles décrivent correctement l'invariant d'un **tas binaire min (min-heap)** ?

Cochez les cases correspondantes aux affirmations correctes :

La priorité de chaque noeud est inférieure ou égale à celles de ses enfants

X

Le tas est un arbre binaire de recherche

L'élément de plus grande priorité (c'est-à-dire ayant la valeur de priorité minimale) se trouve toujours à la racine

X

Les éléments sont stockés dans un tableau trié

Tous les sous-arbres sont eux-mêmes des tas

X

(2 points) Question 2 : On insère successivement n paires (priorité, élément) dans un tas-min (min-heap) initialement vide, puis on les extrait tous.

Quelle est la complexité totale ?

$O(n \log n)$

(4 points) Question 3 : Pourquoi, si l'on connaît à l'avance les éléments à insérer dans une file avec priorité avec des insertions et des extractions fréquentes, un tableau trié périodiquement peut être un meilleur choix qu'un tas-min (min-heap) ?

Parce que lorsque les éléments sont connus à l'avance, on peut les trier globalement (éventuellement en temps linéaire si les priorités sont bornées), ce qui évite les coûts logarithmiques répétés des insertions et extractions dans un heap.

Barème suggéré :

- Mention de "connaître à l'avance" → tri global possible : **1 point**
- Mention d'un gain algorithmique (ex. linéaire, amorti, éviter $\log n$) : **1 point**

(3 points) Question 4 : Considérez un tas-min (heap-min) stocké dans un tableau. Lesquelles des affirmations suivantes sont vraies ?

Cochez les cases correspondantes aux affirmations correctes :

Le tableau est trié

Les éléments sont partiellement ordonnés

La deuxième plus grande priorité est toujours un enfant de la racine

Il existe une feuille qui contient la plus grande valeur de priorité

La racine contient une plus petite valeur de priorité

Les enfants d'un noeud ont des priorités consécutives, l'une suit l'autre dans l'ordre d'extraction

Aucune des affirmations ci-dessus n'est correcte

Toutes les affirmations ci-dessus sont correctes

X
X
X

(4 points) Question 5 : On implémente une `HeapAdaptablePriorityQueue`, une file avec priorité à priorités adaptables (on peut modifier la priorité d'un élément déjà présent avec l'opération `replacePriority( e, p )`) où chaque élément mémorise son index courant dans le tableau du tas-min. Donnez les grandes étapes de l'opération `replacePriority( e, p )`, où `e` est l'élément et `p` la nouvelle priorité.

1. **Récupérer l'index `i`** de l'élément `e` (stocké dans `e`).
2. **Remplacer la priorité** de l'entrée à l'index `i` par la nouvelle priorité `p`.
3. **Restaurer l'invariant du tas-min** en réajustant la position avec l'opération **bubble** :
 - si la nouvelle priorité est **plus petite** que celle du parent → faire **nager** à partir de `i` ;
 - sinon → faire **couler** à partir de `i`.
4. **Pendant chaque échange (swap)** effectué par nager ou couler, **mettre à jour les indices** mémorisés dans les éléments échangés.